

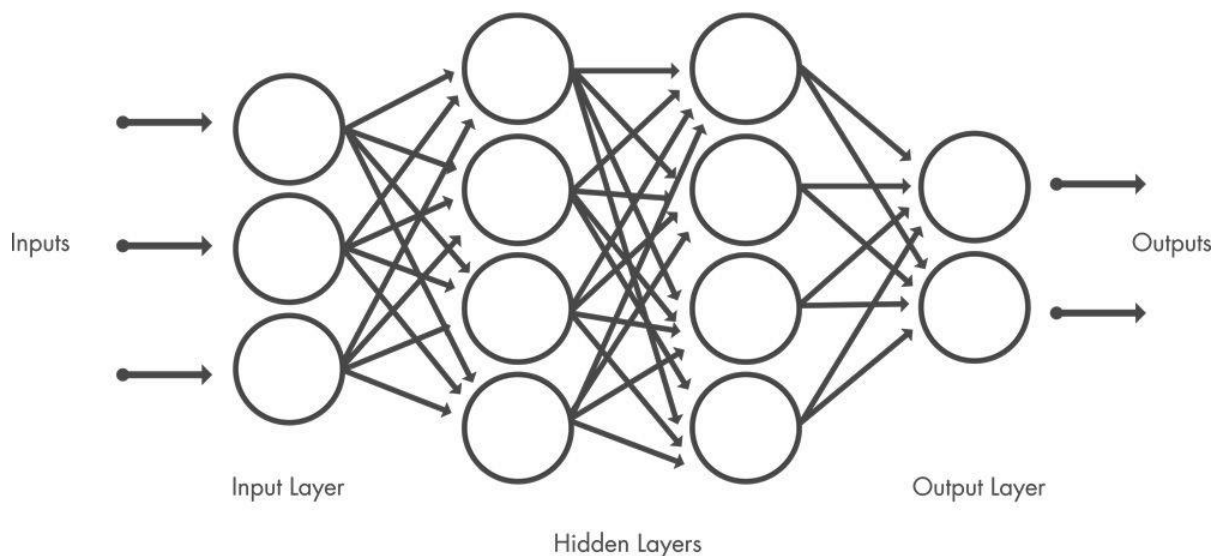
Priprema Lab 10 : Duboko učenje

CNN Konvoluciona neuronska mreža (convolutional neural network- CNN)

Konvoluciona neuronska mreža (convolutional neural network- CNN) može imati desetine ili stotine slojeva od kojih svaki uči da detektuje različite karakteristike slike. Filteri se primjenjuju na svaku sliku za obuku u različitim rezolucijama, a izlaz svake konvolvirane slike se koristi kao ulaz za sljedeći sloj. Filteri mogu početi kao vrlo jednostavne karakteristike, kao što su svjetlina i ivice, i povećati složenost do karakteristika koje jedinstveno definiraju objekt.

Značenje učenja, slojeva i klasifikacije

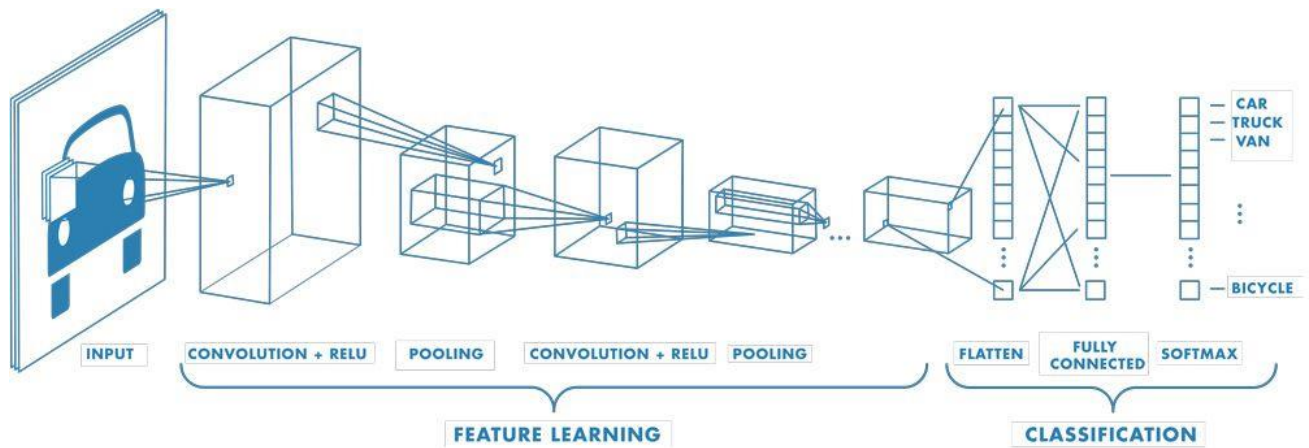
CNN se sastoji od ulaznog sloja, izlaznog sloja i mnogih skrivenih slojeva između.



Ovi slojevi izvode operacije koje mijenjaju podatke s namjerom učenja karakteristika specifičnih za podatke. Tri najčešća sloja su konvolucija, aktivacija ili ReLU i objedinjavanje.

- Konvolucija stavlja ulazne slike kroz skup konvolucionih filtera, od kojih svaki aktivira određene karakteristike slika.
- Rektificirana linearna jedinica (ReLU) omogućava brži i efikasniji trening mapiranjem negativnih vrijednosti na nulu i održavanjem pozitivnih vrijednosti. Ovo se ponekad naziva aktivacija, jer se samo aktivirane karakteristike prenose naprijed u sljedeći sloj.
- Objedinjavanje pojednostavljuje izlaz izvođenjem nelinearnog downsamplinga, smanjujući broj parametara koje mreža treba da nauči.

Ove operacije se ponavljaju u desetinama ili stotinama slojeva, pri čemu svaki sloj uči da identifikuje različite karakteristike.



Primjer mreže s mnogo konvolucijskih slojeva. Filteri se primjenjuju na svaku sliku za obuku u različitim rezolucijama, a izlaz svake konvolvirane slike se koristi kao ulaz za sljedeći sloj.

Zajedničke težine i predrasude

Za razliku od tradicionalne neuronske mreže, CNN ima zajedničke težine i vrijednosti pristranosti, koje su iste za sve skrivene neurone u datom sloju.

To znači da svi skriveni neuroni otkrivaju istu osobinu, kao što je rub ili mrlja, u različitim regijama slike. Ovo čini mrežu tolerantnom na prevođenje objekata na slici. Na primjer, mreža obučena za prepoznavanje automobila moći će to učiniti gdje god se automobil nalazi na slici.

Klasifikacioni slojevi

Nakon učenja karakteristika u mnogim slojevima, arhitektura CNN-a prelazi na klasifikaciju.

Pretposljednji sloj je potpuno povezan sloj koji daje vektor K dimenzija (gdje je K broj klasa koje se mogu predvidjeti) i sadrži vjerovatnoće za svaku klasu slike koja se klasifikuje.

Završni sloj CNN arhitekture koristi klasifikacioni sloj da obezbedi konačni izlaz klasifikacije.

Zašto su CNN-ovi važni

CNN-ovi pružaju optimalnu arhitekturu za otkrivanje i učenje ključnih karakteristika u podacima slika i vremenskih serija. CNN su ključna tehnologija u aplikacijama kao što su:

Medical Imaging

Medicinsko snimanje: CNN može pregledati hiljade izvještaja o patologiji kako bi vizualno otkrio prisustvo ili odsustvo ćelija raka na slikama.

Obrada signala

Obrada zvuka: Detekcija ključne riječi može se koristiti na bilo kojem uređaju s mikrofonom za otkrivanje kada se izgovori određena riječ ili fraza („Hej Siri!“). CNN-ovi mogu precizno naučiti i otkriti ključnu riječ dok ignorišu sve druge fraze bez obzira na okruženje.

Detekcija objekata

Detekcija objekata: Automatizirana vožnja se oslanja na CNN-e da precizno otkrije prisustvo znaka ili drugog objekta i donese odluke na osnovu izlaza.

Sintetičko generiranje podataka

Sintetičko generiranje podataka: Koristeći generativne adversarial mreže (GAN) , nove slike se mogu proizvesti za korištenje u aplikacijama dubokog učenja uključujući prepoznavanje lica i automatiziranu vožnju.

Primjer1 CNN(Konvoluciona neuronska mreža)(demo.m) - Omniglot skup podataka

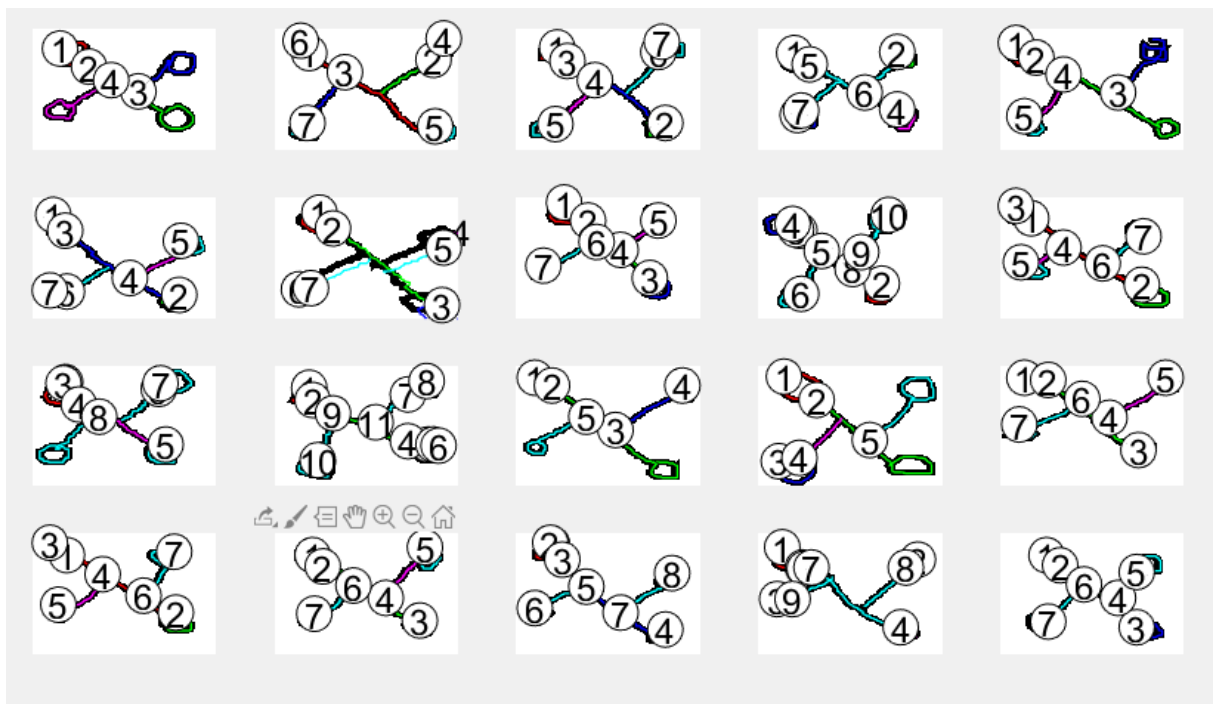
Omniglot skup podataka dizajniran je za razvoj algoritama učenja sličnih ljudima. Sadrži 1623 različita rukom pisana znaka iz 50 različitih abeceda. Svaki od 1623 karaktera nacrtano je online putem Amazon's Mechanical Turk od strane 20 različitih ljudi. Svaka slika je uparena sa podacima o potezima, sekvencama $[x,y,t]$ koordinata sa vremenom (t) u milisekundama.



Omniglot skup podataka sadrži 50 abeceda. Podijelili smo ih u pozadinski skup od 30 abeceda i skup za evaluaciju od 20 abeceda. Da bismo uporedili sa rezultatima u našem radu, samo pozadinski skup treba da se koristi za učenje opšteg znanja o likovima (npr. učenje karakteristika, meta-učenje ili zaključivanje hiperparametara). Jednokratni rezultati učenja se izvještavaju pomoću alfabeta iz skupa za evaluaciju.

Još dva izazovna "minimalna" podjela sadrže samo pet pozadinskih abeceda, označenih kao "pozadina mala 1" i "pozadina mala 2". Ovo je bliža aproksimacija iskustvu koje odrasla osoba može imati za likove općenito. Za cilj izgradnje AI sistema na ljudskom nivou sa minimalnom obukom, s obzirom na grubu procjenu onoga što "minimalno" znači za ljude, postoji potreba da se istraže postavke s manje primjera obuke po razredu i manje osnovnih klasa za učenje za učenje.

Saznajte više o strukturi skupa podataka raspakiranjem direktorija i pokretanjem primjera u matlab. Ovo proizvodi figure poput ove ispod, s različitim bojama koje označavaju različite poteze:



Strukturi skupa podataka pokretanjem skripte 'demo.m' se satoji:

data_background.mat
data_evaluation.mat
data_background_small1.mat
data_background_small2.mat

Da biste uporedili sa rezultatima jednokratne klasifikacije u našem radu, pokrenite 'demo_classification.m' u folderu 'one-shot-classification' da biste demonstrirali osnovni model koristeći modifikovanu Hausdorff Distance.

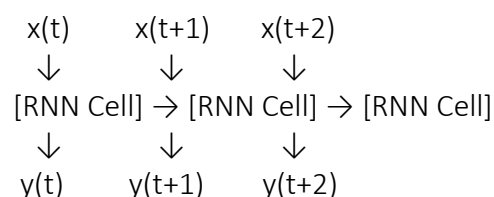
RNN (rekurentne neuronske mreže)

RNN (Recurrent Neural Network) je vrsta neuronske mreže posebno dizajnirana za obradu sekvencijalnih podataka, kao što su:
vremenske serije (temperatura, cijene, EKG...),
tekst i govor,
muzika, video (redoslijed slika/zvuka)

RNN omogućava da informacije "teku kroz vrijeme", tj. koristi prošli kontekst da utiče na trenutni izlaz. Osnovna struktura RNN:

Za svaki trenutak t , mreža obrađuje:

- **ulaz** $x(t)$
- **skriveni kontekst** $h(t-1)$



Svaka ćelija pamti prethodno stanje i koristi ga za dalju obradu.

Evo nekoliko ključnih primjena RNN-a u stvarnom svijetu:

1. Predikcija vremenskih serija

RNN je vrlo koristan u zadacima kao što su predikcija vremenskih serija, gdje imamo podatke koji se razvijaju kroz vrijeme, a želimo predvidjeti buduće vrijednosti na temelju prošlih podataka.

Cijene dionica: Predikcija kretanja cijena dionica na temelju prošlih podataka.

Energetska potrošnja: Prognoza potrošnje struje ili plina na temelju prošlih obrazaca.

Meteorologija: Predikcija vremena (temperatura, padavine) na temelju prošlih vremenskih podataka.

2. Obrada prirodnog jezika (NLP)

RNN su vrlo popularne u obradi prirodnog jezika (NLP) jer mogu obraditi tekst kao sekvencijalne podatke i razumjeti kontekst prošlih riječi.

Prepoznavanje govora: Korištenje RNN-a za prepoznavanje riječi iz audio zapisa (npr. prepoznavanje govora za virtualne asistente poput Siri ili Alexa).

Generisanje teksta: Automatsko generisanje teksta na temelju početne rečenice (npr. generisanje pjesama ili pisanje priča).

Prevodilački modeli: Prevodjenje teksta sa jednog jezika na drugi (npr. Google Translate koristi RNN za prevođenje).

Sentiment analiza: Prepoznavanje sentimenta u tekstu (pozitivan, negativan, neutralan).

3. Prepoznavanje govora i zvuka

RNN je idealan za obradu govora jer može obraditi zvuk i foneme u vremenskim sekvencama, omogućujući prepoznavanje govora ili zvučnih uzoraka.

Prepoznavanje ključnih riječi: Identifikacija određenih fraza ili ključnih riječi kao što je "Hej Siri" ili "OK Google".

Prepoznavanje govora: Prepoznavanje cijelih rečenica i njihova pretvorba u tekst.

Prepoznavanje emocija u govoru: Analiza intonacije i prepoznavanje emocija (sretan, ljut, tužan).

4. Robotika i kontrola sistema

RNN se koristi u robotici za modeliranje i kontrolu dinamičnih sistema koji se razvijaju kroz vrijeme.

Kontrola robota: Korištenje RNN-a za kontrolu kretanja robota, kako bi on mogao učiti iz prošlih akcija i odlučivati kako će se ponašati u budućnosti.

Autonomna vozila: RNN se koristi za predikciju ponašanja na cesti i donošenje odluka o ubrzanju, kočenju ili skretanju na temelju prošlih podataka.

5. Generisanje muzike

RNN može generirati muziku pomoću sekvencijalnih podataka o notama ili akordima.

Muzika: RNN može naučiti obrasce u muzici (npr. klasičnu muziku, pop, jazz) i generirati nove melodije.

Muzika u realnom vremenu: Generisanje muzike u stvarnom vremenu temelji se na prethodnim tonovima, akordima i melodijama.

6. Biomedicina i analiza signala

RNN se također primjenjuju u biomedicinskim istraživanjima za analizu bioloških podataka.

EKG analiza: Detekcija abnormalnosti u elektroencefalogramima (EEG) i elektrokardiogramima (EKG) pomoću RNN-a.

Predikcija zdravstvenih stanja: Korištenje RNN za praćenje i prognozu zdravstvenih stanja na temelju vremenskih serija medicinskih podataka.

7. Video analiza i prepoznavanje objekata

RNN se koristi za analizu video zapisa u realnom vremenu, gdje se mora analizirati sekvencijalni tok slika.

Prepoznavanje objekata: Identifikacija objekata i radnji u video zapisima (npr. prepoznavanje ljudi, vozila, aktivnosti).

Video klasifikacija: Prepoznavanje specifičnih scena u video zapisima na temelju prethodnih okvira (npr. sportski video, filmovi).

8. Predikcija u industriji i proizvodnji

RNN je koristan za predikciju u industrijskim procesima gdje se podaci razvijaju kroz vrijeme, kao što su proizvodni procesi i predikcija kvara opreme.

Predikcija kvarova opreme: Korištenje RNN za analizu podataka sa senzora na mašinama i predviđanje kada bi moglo doći do kvara.

Optimizacija proizvodnih linija: Praćenje i predviđanje tokova u proizvodnji kako bi se optimizirali procesi.

Primjer 2(RNN) - primjena rekurentne neuronsku mrežu (RNN) za predikciju kvarova opreme na temelju podataka sa senzora u MATLAB-u, možemo koristiti simulirane podatke (npr. podatke o vibracijama, temperaturi, brzini itd.) koji se prikupljaju sa senzora na mašinama.

Pretpostavljamo da imamo vremensku seriju podataka sa senzora na mašini, a cilj nam je da predvidimo kada će doći do kvara na osnovu tih podataka.

Kratki opis primjene:

Podaci sa senzora: Na temelju podataka prikupljenih u prethodnim vremenskim koracima, želi se predvidjeti mogućnost kvara.

Obrada podataka: Prethodni podaci se koriste za treniranje modela.

Predikcija: Na temelju modela, predviđamo kada bi mogla doći neispravnost.

ovom primjeru, koristićemo simulirane podatke o vibracijama mašine i temperaturi kao ulazne podatke. Nakon toga ćemo koristiti RNN za predikciju mogućih kvarova. Za ovaj primjer, pretpostavimo da imamo 1000 vremenskih koraka podataka sa senzora. Simulirat ćemo podatke vibracija i temperature, gdje će doći do "kvara" na mašini nakon 800. koraka (npr. nakon 800 dana).

Matlab.

```
clear; clc; close all;
```

```
% Simulacija podataka (vibracije, temperatura)
steps = 1000; % broj vremenskih koraka
t = 1:steps;
```

```
% Simuliramo vibracije i temperaturu
vibration = 0.2 * sin(2*pi*0.05*t) + 0.1 * randn(1, steps); % sinusni
obrazac + šum
temperature = 25 + 5 * sin(2*pi*0.03*t) + 0.5 * randn(1, steps); % sinusni
obrazac + šum
```

```
% Simulacija kvara: Simuliramo kvar od 800. koraka
failure = zeros(1, steps);
failure(801:end) = 1; % Kvar počinje od 801. koraka
```

```

% Prikaži podatke
figure;
subplot(2, 1, 1);
plot(t, vibration);
title('Simulirane vibracije mašine');
xlabel('Vremenski korak');
ylabel('Vibracije');

subplot(2, 1, 2);
plot(t, temperature);
title('Simulirana temperatura mašine');
xlabel('Vremenski korak');
ylabel('Temperatura [°C]');

% Priprema podataka
data = [vibration; temperature]; % Ulazni podaci (vibracije + temperatura)
target = failure; % Cilj (da li je došlo do kvara ili ne)

% Pretvaranje u sekvence (priprema za RNN)
X = con2seq(data); % Ulazni podaci u sekvence
T = con2seq(target); % Cilj u sekvence

% Kreiranje Elmanove rekurentne mreže (ERNN)
inputDelays = 1:3; % Koristimo 3 vremenska kašnjenja
hiddenLayerSize = 10; % 10 neurona u skrivenom sloju
net = layrecnet(inputDelays, hiddenLayerSize);

% Priprema podataka za treniranje
[Xs, Xi, Ai, Ts] = preparets(net, X, T);

% Trening mreže
net.trainFcn = 'trainlm'; % Levenberg-Marquardt optimizator
net.trainParam.showWindow = false; % Isključi prikaz prozora tokom
treniranja
net = train(net, Xs, Ts, Xi, Ai);

% Kreiranje Elmanove rekurentne mreže (ERNN)
inputDelays = 1:3; % Koristimo 3 vremenska kašnjenja
hiddenLayerSize = 10; % 10 neurona u skrivenom sloju
net = layrecnet(inputDelays, hiddenLayerSize);

% Priprema podataka za treniranje
[Xs, Xi, Ai, Ts] = preparets(net, X, T);

% Trening mreže
net.trainFcn = 'trainlm'; % Levenberg-Marquardt optimizator
net.trainParam.showWindow = false; % Isključi prikaz prozora tokom
treniranja
net = train(net, Xs, Ts, Xi, Ai);

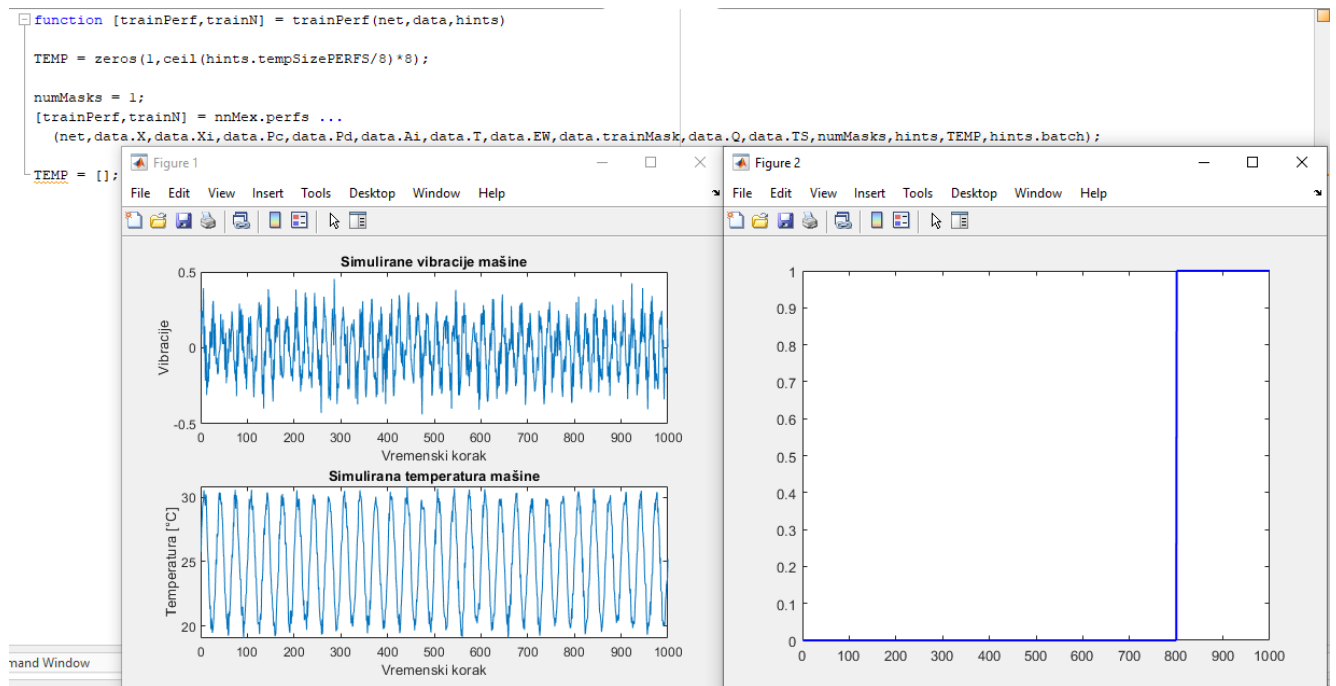
% Predikcija
Y = net(Xs, Xi, Ai);

% Konverzija iz cell u vektor
predicted_failure = cell2mat(Y);

% Prikazivanje stvarne i predviđene vrijednosti
figure;
plot(t, failure, 'b', 'LineWidth', 1.5); hold on; % Stvarna vrijednost
plot(t, predicted_failure, 'r--', 'LineWidth', 1.5); % Predikcija RNN-a
legend('Stvarni kvar', 'Predikcija RNN');
xlabel('Vremenski korak');

```

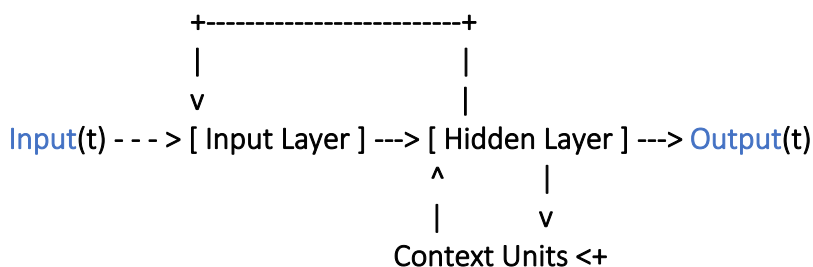
```
ylabel('Kvar (1 = Kvar, 0 = Nema kvara)');
title('Predikcija kvarova na mašini sa RNN');
grid on;
```



ERNN (Elmanove rekurentne neuronske mreže)

Jeffrey Elman (1990) je predstavio model elmanove mreže i jedan je od prvih tipova rekurentnih neuronskih mreža (RNN). Osnovna ideja je da mreža ne koristi samo trenutni ulaz, već i informaciju iz prethodnih stanja putem kontekstnog sloja. Rekurentna se odnosi da skriveni sloj koristi vlastiti prethodni izlaz kao ulaz, dakle mreža ima memoriju i može modelirati sekvence (nizove). Time omogućava pamćenje vremenskih zavisnosti, što je ključno kod vremenskih serija, govora, teksta itd. ERNN koristi kontekstne jedinice koje omogućuju da mreža pamti prethodne informacije, što je ključno kod sekvencijalnih podataka.

Struktura ERNN:



Primjene ERNN (Elmanove RNN) :

1. Predikcija vremenskih serija
Finansijski podaci: cijene dionica, valutni kursevi
Meteorologija: predikcija temperature, padavina
Energetika: potrošnja struje/gasa
2. Obrada prirodnog jezika

Analiza sentimenta
Predikcija sljedeće riječi u rečenici
Prepoznavanje namjere korisnika

3. Biomedicina

Analiza EKG/EEG signala
Detekcija aritmija (predikcija stanja srca)
Predikcija nivoa šećera u krvi

4. Kontrola i robotika

Modeliranje dinamike sistema
Upravljanje autonomnim vozilima

5. Obrada zvuka i govora

Prepoznavanje govora
Detekcija govornika
Predikcija narednih fonema u audio sekvenci

Primjer3. (ERNN1.m) ERNN (Elmanove rekurentne neuronske mreže)

Evo jednostavnog primjera ERNN (Elmanove rekurentne neuronske mreže) u MATLAB-u, sa kodom i grafikom.

Koristit ćemo sintetički vremenski niz (npr. sinusoidu sa šumom) da pokažemo kako ERNN uči i predviđa vrijednosti.

Matlab:

```
% Očisti sve
clear; clc; close all;

% Generiši podatke - sinusoid sa šumom
t = 0:0.1:20;
x = sin(t) + 0.1 * randn(size(t));
X = con2seq(x); % Pretvori vektor u sekvencu (cell array)

% Ciljna vrijednost je ista kao ulaz (samo-prognoza)
T = X;

% Definiši Elmanovu mrežu (layrecnet koristi rekurziju u skrivenom sloju)
inputDelays = 1:2; % Kašnjenja ulaza (npr. 2 vremenska koraka)
hiddenLayerSize = 10; % Broj neurona u skrivenom sloju
net = layrecnet(inputDelays, hiddenLayerSize);

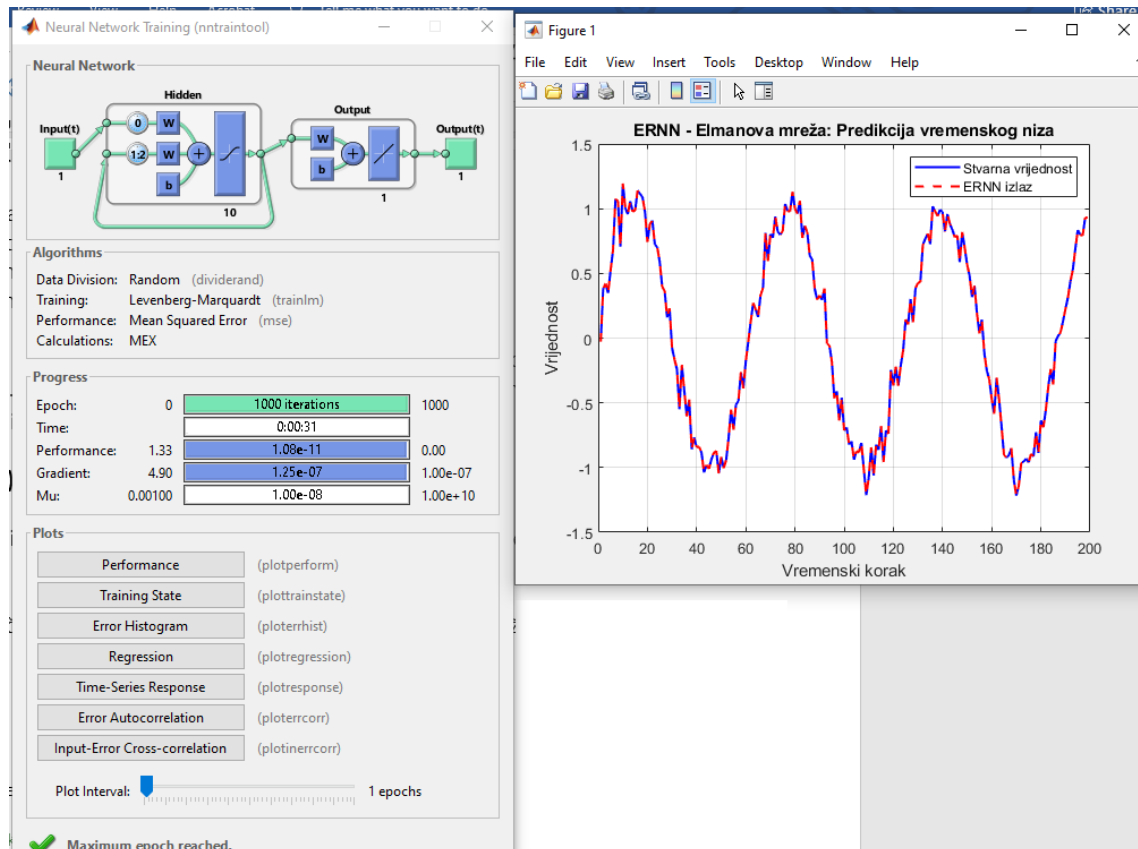
% Priprema podataka za treniranje
[Xs, Xi, Ai, Ts] = preparets(net, X, T);

% Postavke treniranja
net.trainFcn = 'trainlm'; % Levenberg-Marquardt algoritam
net.trainParam.showWindow = true; % Prikaz GUI prozora za treniranje

% Treniraj mrežu
net = train(net, Xs, Ts, Xi, Ai);

% Simuliraj mrežu (predviđanje)
Y = net(Xs, Xi, Ai);
```

```
% Prikaži rezultate - graf
figure;
plot(cell2mat(Ts), 'b', 'LineWidth', 1.5); hold on;
plot(cell2mat(Y), 'r--', 'LineWidth', 1.5);
legend('Stvarna vrijednost', 'ERNN izlaz');
title('ERNN - Elmanova mreža: Predikcija vremenskog niza');
xlabel('Vremenski korak');
ylabel('Vrijednost');
grid on;
```



Primjer 4.(ERNN2) - Predikcija buduće temperature na osnovu prethodnih vrijednosti

U primjeru se pokazuje kako se vrši generisanje (ili učitavanje) vremenskog niza temperature, zatim vrši kreiranje i treniranje Elmanove mreže, kako se koristi predikcija i prikaz rezultata.

Matlab:

```
clear; clc; close all;
```

```
% Simulirani temperaturni podaci (npr. dnevna temperatura)
days = 1:100;
temperature = 20 + 5*sin(2*pi*days/30) + 0.5*randn(1, 100); % periodična
sinusoidna promjena + šum
```

```
% Pretvaranje u sekvencu (za RNN)
X = con2seq(temperature(1:end-1)); % Ulaz (sve osim zadnje)
T = con2seq(temperature(2:end)); % Cilj (sve osim prve, pomaknuto za 1
dan)
```

```
% Definiši ERNN (Elmanova rekurentna mreža)
inputDelays = 1:2;
```

```

hiddenLayerSize = 10;
net = layrecnet(inputDelays, hiddenLayerSize);

% Priprema podataka
[Xs, Xi, Ai, Ts] = preparets(net, X, T);

% Postavke treniranja
net.trainFcn = 'trainlm'; % Levenberg-Marquardt
net.trainParam.showWindow = true;

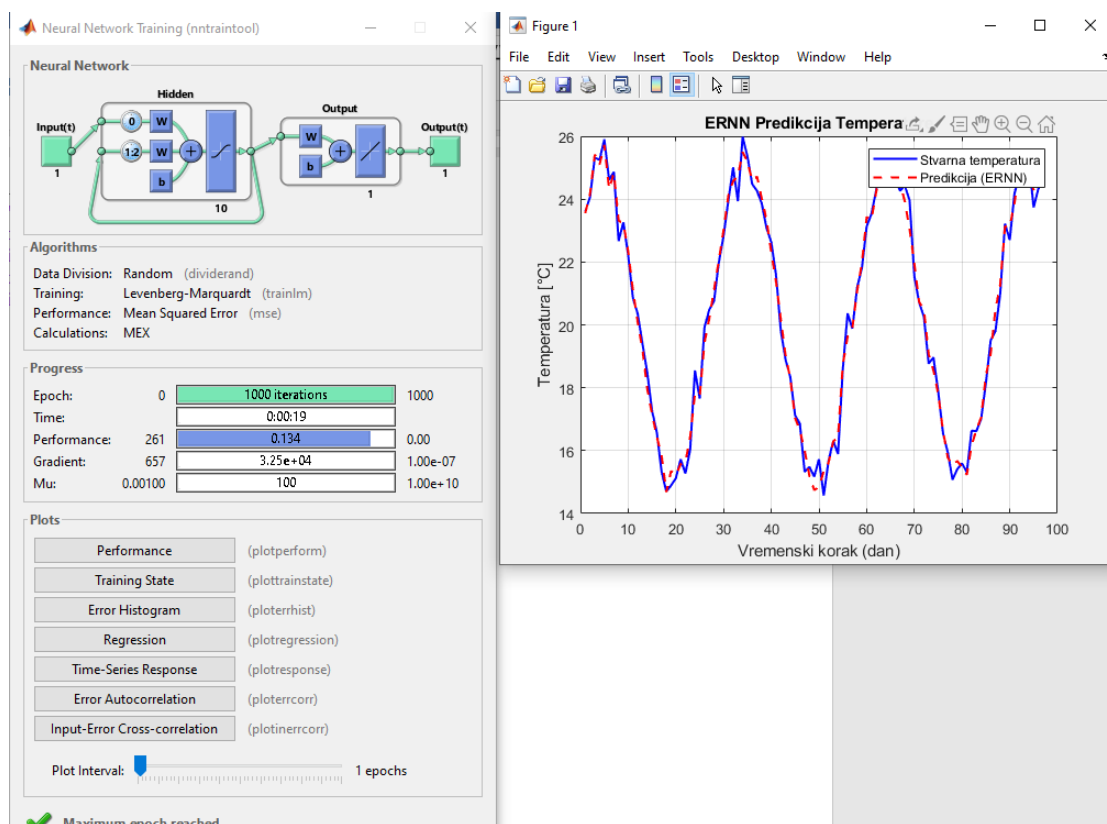
% Treniraj mrežu
net = train(net, Xs, Ts, Xi, Ai);

% Predikcija
Y = net(Xs, Xi, Ai);

% Konverzija iz cell u vektor
actual = cell2mat(Ts);
predicted = cell2mat(Y);

% Graf
figure;
plot(actual, 'b', 'LineWidth', 1.5); hold on;
plot(predicted, 'r--', 'LineWidth', 1.5);
legend('Stvarna temperatura', 'Predikcija (ERNN)');
xlabel('Vremenski korak (dan)');
ylabel('Temperatura [°C]');
title('ERNN Predikcija Temperature');
grid on;

```



Na grafu se mogu vidjeti plava i crvena linija:
 plava linija: stvarna temperatura (sinusoida + šum)
 crvena isprekidana linija: predikcija ERNN-a

Ako mreža uči dobro, predikcija će pratiti trend sinusoida.